

```
1 #%% md
2 # Assignment 7.10 Database Monitoring
3 #%% md
4 I am using Postgres (allowed as per Professor Gridiron) and I
  installed `pg-view` and `pg-activity` on Ubuntu (ugali is my
  computer name):
5 ```
6 pip3 install pg_view pg_activity
7 ```
8 #%% md
9 I then ran `pg_activity` which brings up some basic stats on
  your database like the uptime, size, number of sessions/
  processes and some other basic info.<p>
10 I noticed that the size does not seem to be accurate; I will
  investigate to find out why.
11 ```bash
12 pg_activity -h ugali -U postgres -d postgres
13 ```
14 As I ran this query below, I took a screenshot while it shows
  the active query; you can see I ran the query from DBeaver, an
  open source Query tool that can be used with Postgres, SQL
  Server and possibly a hundred additional databases.
15 #%% md
16 ```SQL
17 SELECT * FROM put_quote ORDER BY day_seq,etl_time_pac desc;
18 ```
19 ![image.png](attachment:cea8ac18-4a0c-4f52-be79-64d9991da4a4.
  png)
20 #%% md
21 It turns out that pg_view is an older, less maintained version
  and it did not work with my version of Postgres. But it looks
  like pg_activity has all the same major features.
22 #%% md
23 I restarted `pg_activity` with the switch `--db-size` and now
  it shows the database size:<p>
24 ![image.png](attachment:1f5e9b1a-2e99-4772-ace3-5e4399854041.
  png)<p>
25 Size is not calculated by default because it likely has a CPU
  or disk overhead; so the user needs to affirmatively pass that
  argument.
```

```
26
27 #%% md
28 One big potential issue with Databases is when multiple users
   try to write and read the same data at the same time. I am
   going to simulate a "dead lock" where one user is trying to
   write data and another user tries to read that date before it
   is written to see if we can learn anything from `pg_activity`.
29 #%% md
30 ```sql
31 -- Example 1: Row-level lock using FOR UPDATE
32 -- Setup (run once)
33 DROP TABLE IF EXISTS inventory;
34 CREATE TABLE inventory (id serial PRIMARY KEY, qty int);
35 INSERT INTO inventory (qty) VALUES (10);
36
37 -- Session A:
38 BEGIN;
39 -- acquires a row lock on id=1 and holds it until COMMIT/
   ROLLBACK
40 SELECT * FROM inventory WHERE id = 1 FOR UPDATE;
41
42 -- (keep this transaction open)
43
44 -- Session B (will block until Session A commits/rolls back):
45 BEGIN;
46 -- this UPDATE needs the same row lock and will wait
47 UPDATE inventory SET qty = qty - 1 WHERE id = 1;
48 -- once Session A COMMITS, this UPDATE will complete
49 ```
50 Now this is what `pg_activity` looks like:
51 ![image.png](attachment:652bb923-5bf2-4855-9c09-291667278a5e.
   png)
52
53 Normally, there are no queries listed here unless it is a long
   running query; the ones above are very small, but there is a
   dead lock situation where this will never complete if I don't
   end one of the sessions.
54 #%% md
55 ```sql
56 -- To release the lock in Session A:
```

```
57 COMMIT; -- then Session B's UPDATE proceeds
58 ```
59 I just committed the above transaction.<p>
60 ![image.png](attachment:63f36c88-a203-40f2-8412-a57c4da7b2c1.
    png)
61
62 Finally, I commit Session B and all the queries dissapper:<p>
63 ![image.png](attachment:a0f53a26-c5b3-4dba-85da-b9a586ddff04.
    png)
```

```
64 #%% md
65 ## Conclusion
66 pg_activity is a lightweight, practical tool for interactive
    monitoring:
67 - it quickly reveals active queries, blocking relationships,
    and (with --db-size) database size.
68 - pg_view should be avoided on newer Postgres versions;
    prefer pg_activity.
69 - the locking examples validated that long-lived transactions
    produce visible blocking in pg_activity, making it useful for
    diagnosing concurrency issues.
```